LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL [6] – Inheritance



Rimbhi Putri Aulia Azzahra

103132400018

PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Inhertance (pewarisan) sebuah class child/subclass nya mewarisi atribut dan method dari parent atau superclassnya. Inheritance bukan hanya sekedar fitur untuk menghemat pengetikan kode, Inheritance mekanisme pemodelan konseptual yang membawa konsekuensi terhadap struktur, keterbacaan, dan evolusi sebuah sistem perangkat lunak.

B. Guided

1. Guided01

- A.java

```java
package com.example.rimbhi.guided.guided01;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+x+", y = "+y);
    }
}
```

- B.java

```java
package com.example.rimbhi.guided.guided01;

public class B extends A{
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = (x+y+z);

        System.out.println("Jumlah (x+y+z) = " + hasil );
    }

}
```

- Demo.java

```java
package com.example.rimbhi.guided.guided01;

public class Demo {
    public static void main(String[] args){

        A superclass = new A();
```

```
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("Subclass: ");
        subclass.jumlahXYZ();



    }
}
```

Output:

```
Superclass:
Nilai x = 3, y = 4
Subclass:
Jumlah (x+y+z) = 8
```

2. Guided02
   - Employe.java

```
package com.example.rimbhi.guided.guided02;

public class Employe {
    private String nama;
    String departemen;

    public Employe(String nama){
        this.nama = nama;
    }

    public void tampilNama(){
        System.out.println("nama: "+ nama);
    }
}
```

   - Manager.java

```
package com.example.rimbhi.guided.guided02;

public class Manager extends Employe{
```

```
    private  String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: "+alamat);
        System.out.println("departemen: "+
super.departemen);

    }

}
```

- Demo.java

```
package com.example.rimbhi.guided.guided02;

public class Demo {
    public static void main(String[] args){
        Manager adi = new Manager("Adi", "Sukabiru");
        adi.departemen = "SDM";
        adi.tampilInformasi();

    }

}
```

Output:

```
nama: Adi
alamat: Sukabiru
departemen: SDM
```

3. Guided03
   - Lagu.java

```
package com.example.rimbhi.guided.guided03;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1){
        judul = param1;
        pencipta = "Tidak dikenal";
    }
```

```
    void IsiParam(String param1, String param2){
        judul = param1;
        pencipta = param2;
    }

    void tampilkan(){
        System.out.println("Judul: "+ judul + "Pencipta: "+
pencipta);
    }
}
```

- Main.java

```
package com.example.rimbhi.guided.guided03;

public class Main {
    public static void main(String[] args){
        Lagu d, e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1 ");
        e.IsiParam("Lagu 2", "aku");

        d.tampilkan();
        e.tampilkan();
    }
}
```

Output:

```
Judul: Lagu 1 Pencipta: Tidak dikenal
Judul: Lagu 2Pencipta: aku
```

4. Guided04
   - A.java

```
package com.example.rimbhi.guided.guided04;

public class A {
    public void tampilkan(){
         System.out.println("Method milik class A dipanggil
...");
    }
}
```

   - B.java

```java
package com.example.rimbhi.guided.guided04;

public class B extends A {
    public void tampilkan(){
        super.tampilkan();
        System.out.println("Method milik class B dipanggil
...");
    }

}
```

- Main.java

```java
package com.example.rimbhi.guided.guided04;

public class Main {
    public static void main(String[] args){
        B overideB = new B();
        overideB.tampilkan();
    }
}
```

Output:

```
Method milik class A dipanggil ...
Method milik class B dipanggil ...
```

C. Unguided

1. Soal Unguided

Buatlah sebuah program yang memodelkan estimasi biaya komputasi dari tigas jenis model machine learning. Dengan ketentuan utamanya, semua atribut di dalam kelas harus bersifat private, sementara untuk penamaan kelas, atribut, dan method diberikan kebebasan.

Pertama, buatlah Kelas Induk yang menyimpan atribut nama model (String) dan jumlah data latih (int). Kelas ini harus memiliki *constructor* untuk menginisialisasi kedua nilai tersebut, sebuah *getter* agar kelas lain dapat membaca jumlah data latih, serta method untuk menampilkan informasi. Kelas induk juga wajib memiliki dua method hitung biaya yang menerapkan *overloading*: method tanpa parameter yang menghitung jumlah data latih × 2, dan method dengan satu parameter (jumlah *epoch*) yang menghitung jumlah data latih × epoch.

Selanjutnya, buatlah Kelas Turunan Pertama (Model Regresi) yang mewarisi kelas induk dan menambahkan atribut jumlah fitur. *Constructor*-nya harus menerima nama, data latih, dan fitur dengan memanggil *constructor* kelas induk. Di kelas ini,

harus meng-*override* method tampil informasi dan method hitung biaya tanpa parameter (dengan rumus biaya kelas induk + (jumlah fitur × 10)). Selain itu, tambahkan satu method baru yang merupakan *overload* ketiga, yaitu method hitung biaya dengan dua parameter (jumlah *epoch* dan ukuran *batch*) yang menghitung (jumlah data latih × epoch) / ukuran batch.

Kemudian, buat Kelas Turunan Kedua (Model Klasifikasi) yang mewarisi kelas induk dan menambahkan atribut jumlah kelas. *Constructor*-nya menerima nama, data latih, dan jumlah kelas. Kelas ini juga harus meng-*override* method tampil informasi dan method hitung biaya tanpa parameter (dengan rumus biaya kelas induk × jumlah kelas). method hitung biaya dengan parameter *epoch* tidak boleh di-*override* di kelas ini. Setelah itu, buat Kelas Turunan Ketiga (Model Deep Learning) yang mewarisi Kelas Turunan Kedua dengan menambahkan atribut jumlah layer. *Constructor*-nya menerima nama, data latih, jumlah kelas, dan jumlah layer dengan memanggil *constructor* kelas di atasnya. Kelas ini wajib meng-*override* method tampil informasi dan method hitung biaya tanpa parameter (dengan rumus biaya kelas di atasnya + (jumlah layer × 200)). Terakhir, buat Kelas Main yang menginstansiasi tiga objek: objek Regresi bernama "Regresi Linier" dengan 500 data latih dan 4 fitur; objek Klasifikasi bernama "Decision Tree" dengan 800 data latih dan 3 kelas; serta objek Deep Learning bernama "Neural Network" dengan 1000 data latih, 5 kelas, dan 3 layer. Ketiga objek tersebut harus disimpan ke dalam sebuah array bertipe kelas induk. Gunakan perulangan untuk menampilkan informasi model, menampilkan biaya komputasi, dan menjumlahkan seluruh biaya dari setiap elemen array. Setelah perulangan selesai, tampilkan total biaya secara keseluruhan. Terakhir, cetak tiga informasi tambahan di bagian paling bawah: biaya 10 *epoch* untuk model "Neural Network" (diambil dari array), biaya 10 *epoch* untuk model "Regresi Linier", dan biaya 10 *epoch* dengan ukuran *batch* 50 untuk model "Regresi Linier".

- Ketua.java

```
package com.example.rimbhi.unguided;

public class Ketua {
    private String namaModel;
    private int jumlahDataLatih;

    public Ketua(String namaModel, int jumlahDataLatih){
```

```
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public int getjumlahDataLatih(){
        return jumlahDataLatih;
    }

    public String getNamaModel(){
        return namaModel;
    }

    public void tampilInfo(){
        System.out.println("Model: " + namaModel + " | Data
latih: " + jumlahDataLatih);
    }

    public int hitungBiaya(){
        return jumlahDataLatih * 2;
    }

    public int hitungBiaya(int epoch){
        return jumlahDataLatih * epoch;
    }
}
```

- Anggota1.java

```
package com.example.rimbhi.unguided;

public class Anggota1 extends Ketua {

    private int jumlahFitur;

    public Anggota1(String namaModel, int jumlahDataLatih,
int jumlahFitur){
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    public int hitungBiaya(){
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    public int hitungBiaya(int epoch, int ukuranBatch) {
        return (getjumlahDataLatih() * epoch) / ukuranBatch;
    }

    public void tampilInfo() {
```

```
        super.tampilInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }
}
```

- Anggota2.java

```
package com.example.rimbhi.unguided;

public class Anggota2 extends Ketua {

    private int jumlahKelas;

    public Anggota2(String namaModel, int jumlahDataLatih,
int jumlahKelas){
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    public void tampilInfo(){
        super.tampilInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    public int hitungBiaya(){
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

- Anggota3.java

```
package com.example.rimbhi.unguided;

public class Anggota3 extends Anggota2 {

    private int jumlahLayer;

    public Anggota3(String namaModel, int jumlahDataLatih,
int jumlahKelas, int jumlahLayer){
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    public void tampilInfo(){
        super.tampilInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    public int hitungBiaya(){
```

```
        return super.hitungBiaya() + (jumlahLayer * 200);
    }

}
```

- Main.java

```
package com.example.rimbhi.unguided;

public class Main {
    public static void main(String[] args){
        Anggota1 regresi = new Anggota1("Regresi Linier",
500, 4);
        Anggota2 klasifikasi = new Anggota2("Decision Tree",
800, 3);
        Anggota3 deepLearning = new Anggota3("Neural
Network", 1000, 5, 3);

        Ketua[] daftarModel = {regresi, klasifikasi,
deepLearning};

        int totalBiaya = 0;

        for (Ketua model : daftarModel){
        model.tampilInfo();
        int biaya = model.hitungBiaya();
        System.out.println("Biaya komputasi: " + biaya);
        System.out.println("---");
        totalBiaya += biaya;
    }

        System.out.println("Total biaya: " + totalBiaya);
        System.out.println("Biaya 10 epoch (Neural Network):
" + deepLearning.hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier):
" + regresi.hitungBiaya(10));
        System.out.println("Biaya 10 epoch batch 50 (Regresi
Linier): " + regresi.hitungBiaya(10, 50));

    }
}
```

Penjelasan:

- Kelas ketua ini sebagai kelas induk dengan access modifier nya private yang berartihanya boleh diakses dalam kelas itu sendiri, pada kelas ini terdiri

constructor yang digunakan untuk mengisi nilai atribut yang private, selain itu ada juga getter yang digunakan untuk memanggil super.getJumlahDataLatih atau untuk memanggil di kelas anak yang lain. Pada kelas ini terdapat overloading di hitungBiaya() dengan yang satu kodong dan yang satu berisi (int epoch).

- Pada kelas Anggota1 kelas turunan pertama yang mewarisi kelas ketua. Kelas ini menambah atribut baru jumlahFitur yang bersifat private. Di constructor nya terdapat pemanggilan super(namaModel, jumlahDataLatih) yang berfungsi untuk mengirim data constructor kelas induk. Kelas ini melakukan Override pada method tampilInfo() dengan memanggil super.tampilInfo() dulu supaya informasi dari induk tetap tercetak, lalu menambahkan baris cetakan untuk jumlah fitur. Selain itu, kelas ini meng-override method hitungBiaya() tanpa parameter, di mana rumusnya memanggil biaya dari kelas induk super.hitungBiaya() lalu ditambahkan dengan (jumlahFitur ×10). Terakhir, kelas ini punya method baru yang merupakan Overload ketiga, yaitu hitungBiaya(int epoch, int ukuranBatch) yang digunakan untuk menghitung biaya.
- Pada kelas Anggota2 kelas turunan kedua yang juga mewarisi (extends) kelas Ketua. Kelas ini menambahkan atribut baru yaitu jumlahKelas yang bersifat private. Sama seperti Anggota1, constructor nya memanggil super(namaModel, jumlahDataLatih) untuk mengisi data ke kelas induk, dilanjutkan dengan mengisi nilai jumlahKelas. Kelas ini meng override method tampilInfo() dengan panggil super.tampilInfo() lalu menambahkan cetakan untuk jumlah kelas. Kelas ini juga meng override method hitungBiaya() tanpa parameter, dengan rumus yang berbeda, yaitu dengan mengalikan biaya dari kelas induk dengan jumlah kelas (super.hitungBiaya() × jumlahKelas). kelas ini tidak meng override method hitungBiaya(int epoch), sehingga jika method tersebut dipanggil pada objek Anggota2, Java otomatis menggunakan rumus milik kelas induk (Ketua).
- Di kelas Anggota3 ini kelas turunan ketiga yang mewarisi (extends) kelas Anggota2 (bukan langsung ke kelas Ketua). Kelas ini menambahkan atribut baru yaitu jumlahLayer yang bersifat private. Constructor-nya memanggil super(namaModel, jumlahDataLatih, jumlahKelas) yang mengarah ke

constructor Anggota2, lalu dilanjutkan dengan menginisialisasi nilai jumlahLayer. Kelas ini meng-override method tampilInfo() dengan memanggil super.tampilInfo() (yang mana method super ini sudah mencakup info dari Ketua dan Anggota2) lalu menambahkan cetakan untuk jumlah layer. Kelas ini juga meng-override method hitungBiaya() tanpa parameter, di mana rumusnya mengambil biaya dari kelas di atasnya super.hitungBiaya() (yaitu biaya milik Anggota2) lalu ditambahkan dengan (jumlahLayer ×200).

- Pada kelas main dilakukan pembuatan tiga objek dari kelas turunan, objek regresi, dari kelas Anggota1, objek klasifikasi dari kelas Anggota2, dan objek deepLearning dari kelas Anggota3. Ketiga objek tersebut kemudian dimasukkan ke dalam sebuah array bertipe kelas induk (Ketua[]), yang merupakan penerapan konsep Polimorfisme. program menggunakan perulangan for each untuk memanggil method tampilInfo() dan hitungBiaya() dari setiap objek di dalam array. Hasil biaya tiap model diakumulasikan ke dalam variabel totalBiaya. Setelah perulangan selesai, program mencetak total biaya keseluruhan. Terakhir, program mencetak tiga output tambahan, yaitu memanggil method hitungBiaya(10) pada objek *Neural Network* dan *Regresi Linier*, serta memanggil method hitungBiaya(10, 50) pada objek *Regresi Linier* untuk menghitung biaya dengan ukuran batch.

Output:

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

D. Kesimpulan

Inheritance memungkinkan sebuah kelas turunan (*subclass*) mewarisi atribut dan *method* dari kelas induk (*superclass*). Hal ini sangat efektif untuk menghindari duplikasi kode dan menciptakan hierarki kelas yang terstruktur. Dalam implementasinya, *keyword* super berperan krusial untuk mengakses anggota kelas induk, baik untuk memanggil *constructor* maupun *method*-nya. Selain itu, pemahaman mengenai Overloading (method dengan nama sama namun parameter berbeda dalam satu kelas) dan Overriding (penulisan ulang method kelas induk di kelas turunan) sangat penting agar program dapat berjalan secara dinamis.